Révisions du contrôle de TD — R3.L01

Représentations et codages des images : tout le cours, les TD1, TD2 et TD3 corrigés, des exercices

Contrôle théorique (TD) : **14/10/2026** — contrôle pratique (TP) : 06/11/2026 (ou 21/10).

Fiche écrite à partir du CM d'E. Remy (version du 27 août 2026), des sujets de TD et des corrigés pris en TD.
Les numéros « CM diapo n » renvoient au PDF du cours.

Ce qui peut tomber, et comment réviser

Le programme

1. **Texte et binaire** : ASCII, UTF-8, fins de ligne, `ios::binary`, `put/get/write/read`.
2. **Données en mémoire** : tailles des types, `<cstdint>`, alignement et padding, **endianness**.
3. **Images bitmap** : tableau à plat, GrayImage, Color, ColorImage.
4. **PGM / PPM** : lire un dump hexadécimal, écrire `writePGM`, `skip_comments`, `readPGM` (TD1, TD2).
5. **TD2** : `swap_bytes`, lecture de données big endian, version portable, tailles mémoire, débits.
6. **TD3** : rééchantillonnage simple et bilinéaire.
7. **Culture du cours** : couleurs, palette, aliasing, quantification, compression (RLE, Huffman, LZW), formats, vectoriel.

Méthode

- Refaire **à la main** les TD1 et TD2 (code sur papier, sans compilateur).
- Savoir **justifier** chaque choix (« pourquoi `static` ? », « pourquoi `ios::binary` ? »).
- S'entraîner aux **calculs** : tailles, débits, taux de compression, conversions hexadécimales.
- Faire les exercices de la fin, puis vérifier avec le corrigé.

Rappels hexadécimal

1 chiffre hexa = 4 bits, 1 octet = 2 chiffres. `0x0a` = 10 (LF), `0x20` = espace, `0x30`-`0x39` = '0'-'9', `0x41` = 'A', `0x61` = 'a', `0xff` = 255, `0x80` = 128.

Fichiers « texte » et fichiers binaires CM diapos 3 à 8

- **Fichier texte** : chaque octet est un caractère (code ASCII). ASCII = **7 bits**, 128 codes, aucun accent. Les 128 codes restants ont servi à des jeux incompatibles : d'où les « problèmes d'accents ».
- **UTF-8** : 1 à 4 octets par symbole ; les 128 premiers codes sont l'ASCII (compatibilité). Défaut : la taille en octets ne donne pas le nombre de caractères.
- **Fin de ligne** : LF (10) sous Unix, Linux, macOS ; CR (13) sur les anciens Mac ; **CR LF (13, 10)** sous Windows. (La diapo 4 écrit « (10,13) » : c'est bien CR = 13 puis LF = 10.)
- **Fichier binaire** : tout le reste. On l'examine avec un **éditeur hexadécimal**, pas un éditeur de texte.
- Texte : portable (XML, JSON...). Binaire : plus **compact** et plus **rapide**.

Le même nombre, deux codages (CM diapo 8)

4779 = 0x12AB

codage	octets	taille
entier 32 bits little endian	AB 12 00 00	4
chaîne "4779" (avec le '\0')	34 37 37 39 00	5

Pourquoi pas << et >> en binaire ?

- `>>` **saute les blancs** : un octet 10 ou 32 dans les données disparaîtrait.
- `<<` écrit du **texte** (320 → "320").
- `endl` n'écrit pas les mêmes octets partout.

Lire et écrire en binaire en C++ CM diapos 9 et 18 à 22

```
std::ofstream ofs("f.dat", std::ios::binary); // 2e argument OBLIGATOIRE
std::ifstream ifs("f.dat", std::ios::binary); // sinon : conversions de fin de ligne
```

méthode	rôle
`os.put(char c)`	écrit 1 octet
`is.get(char& c)`	lit 1 octet (même un blanc)
`os.write(const char* p, n)`	écrit n octets d'un coup
`is.read(char* p, n)`	lit n octets d'un coup
`is.putback(char c)`	remet c dans le flux

Fermeture automatique par le **destructeur** du flux, en fin de bloc `{ }`.

Octet par octet ou par bloc ?

Octet par octet (`put` / `get`) : simple, on choisit l'ordre des octets (donc l'endianness) à la main ; mais **lent**.

Par bloc (`write` / `read`) : très rapide ; mais recopie la mémoire **telle quelle**, padding et endianness compris.

En pratique (CM diapo 22)

Soit du texte (souvent XML) compressé en ZIP : .docx, .odt...

Soit du binaire lu/écrit par blocs, en **respectant le format** et l'endianness qu'il impose : JPEG, TGA, .doc...

```
// cast obligatoire : write attend un const char*
os.write((const char*)&db, sizeof(DataBlock));
```

Tailles des types et pièges CM diapos 10 à 13

type	Win 32	Win 64	Linux 32	Linux 64
char	1	1	1	1
short	2	2	2	2
int	4	4	4	4
long	4	**4**	4	**8**
long long	8	8	8	8
void*, size_t	4	**8**	4	**8**
float / double	4 / 8 partout (norme IEEE 754)			
long double	dépend de la machine (8, 12, 16...)			

Comme les tailles varient, on utilise les types de `<cstdint>` (C++11) : `int8_t` à `int64_t`, `uint8_t` à `uint64_t`. `uint8_t` : [0 ; 255], `uint16_t` : [0 ; 65535], `int16_t` : [−32768 ; 32767].

Les 3 pièges (CM diapo 13)

1. `uint8_t u = 65; cout << u;` affiche **'A'** (c'est un `unsigned char`). Écrire `cout << (unsigned int)u;`
2. Comparer un `signed` et un `unsigned` : −1 < 1u est **faux** (−1 devient 4 294 967 295). Utiliser le même type des deux côtés.
3. Boucle infinie :

```
for (uint8_t i = 0; i <= 255; i++) truc();
```

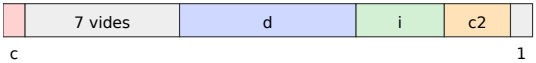
après 255, i++ donne 0 : la condition est toujours vraie.

`signed` et `unsigned` ont la même taille ; seule l'interprétation des bits change.

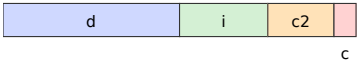
Alignement et remplissage (padding) CM diapos 14, 20

Une donnée de 2, 4 ou 8 octets doit être à une adresse **multiple de sa taille**. Le compilateur ajoute des octets vides ; une structure est alignée sur son membre le plus exigeant et sa taille est un multiple de cet alignement.

noalign { char c; double d; int i; char c2[3]; } → 24 octets



align { double d; int i; char c2[3]; char c; } → 16 octets



1 case = 1 octet ; gris = remplissage

Exemple du CM : DataBlock (diapo 20)

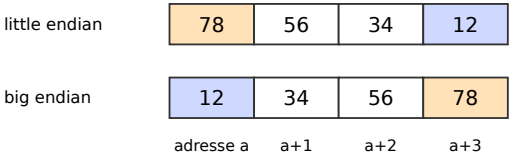
```
struct DataBlock {      // alignement : 4
  uint8_t  alpha[5];    // 5 octets + 3 de padding
  uint32_t bravo[2];    // 2 × 4 = 8 octets
  uint16_t charlie;     // 2 octets + 2 de padding
};                      // 5+3+8+2+2 = 20 octets
```

Conséquences

- L'**ordre** des membres change la taille (24 contre 16).
- Écrire une structure d'un bloc écrit aussi le padding : le fichier ne respecte pas un format qui n'en a pas. Il faut alors lire/écrire **champ par champ**.
- C'est pourquoi on vérifie `sizeof(Color) == 3` avant d'écrire un tableau de `Color` d'un bloc.

Little endian et big endian CM diapos 15 à 19

l'entier 32 bits 0x12345678



- **Little endian** (« petit-boutiste ») : poids **faible** en premier. PC Intel/AMD (Windows, Linux, macOS Intel), iOS sur ARM.
- **Big endian** : poids **fort** en premier. PowerPC, 680x0... et le **réseau** (IP).
- **Bi-endian** : ARM, MIPS (choisi au démarrage).

Le danger : échanger des données entre deux machines différentes sans convertir. L'important n'est pas l'ordre choisi, mais **se mettre d'accord** : chaque format de fichier fixe le sien (TGA : little ; JPEG : big).

CM diapo 18 : écrire en big endian depuis un PC

```
int32_t data = 0x12345678;
const char* ptr = (const char*)&data; // ptr[0] = 0x78 sur PC
std::ofstream os("filename.dat", std::ios::binary);
os.put(*(ptr + 3));   // 0x12 : poids fort d'abord
os.put(*(ptr + 2));   // 0x34
os.put(*(ptr + 1));   // 0x56
os.put(*ptr);         // 0x78
```

On regarde l'entier comme un tableau de 4 octets grâce au pointeur `char*`, puis on les écrit dans l'ordre voulu. La lecture (diapo 19) fait `is.get(*(ptr + 3))` ... dans le même ordre.

TD2 exercice 2, question 1 : swap_bytes

```cpp
#include <cstdint>
#include <cstddef>   // size_t

template <typename T>
void swap_bytes(T& t)
{
  // On regarde t comme un tableau de sizeof(T) octets.
  uint8_t* ptr = reinterpret_cast<uint8_t*>(&t);
  for (size_t i = 0; i < sizeof(T) / 2; i++) {
    uint8_t tmp              = ptr[i];
    ptr[i]                   = ptr[sizeof(T) - 1 - i];
    ptr[sizeof(T) - 1 - i]   = tmp;
  }
}
```

On échange le 1er et le dernier octet, le 2e et l'avant-dernier... jusqu'au milieu : `sizeof(T) / 2` échanges. Ça marche pour `int16_t`, `int32_t`, `double`...

Trace sur un double (8 octets)

i = 0 : échange ptr[0] et ptr[7] ; i = 1 : ptr[1] et ptr[6] ; i = 2 : ptr[2] et ptr[5] ; i = 3 : ptr[3] et ptr[4]. Fin (4 = 8 / 2).

Sur un `int16_t` valant 0x1234, en mémoire PC 34 12 → 12 34 : il vaut maintenant 0x3412.

Erreurs à éviter (vues dans les notes de TD)

- `reinterpret_cast<uint8_t>(&var)` : il faut un type **pointeur**, `<uint8_t*>`.
- `if (*ptr = 0x34)` : `=` est une affectation, il faut `==`.
- Préférer `uint8_t` à `int8_t` pour manipuler des octets bruts (pas de valeurs négatives).
- `struct` : tout est **public** par défaut ; `class` : **private**.

TD2 exercice 2, question 2 : lire data.dat (big endian) sur un PC

```
struct Data {
  int16_t     a;          // entier signé 16 bits
  double      b;          // réel IEEE 754 double précision
  char        fourcc[4];  // exactement 4 caractères, pas de
'\0'
  std::string d;          // chaîne de 0 à 255 caractères
};

void read_Data(Data& data, std::istream& is)
{
  is.read((char*)&data.a, 2);
  is.read((char*)&data.b, 8);
  is.read(data.fourcc, 4);
  uint8_t taille = 0;                    // l'octet de préfixe
  is.read((char*)&taille, 1);
  data.d = std::string(taille, ' ');     // (nombre,
caractère) !
  is.read(&data.d[0], taille);
  if (!is)
    throw std::runtime_error("read_Data : fichier tronqué");
  swap_bytes(data.a);                    // big -> little
endian
  swap_bytes(data.b);                    // (sur un PC little
endian)
}
```

- **Champ par champ**, pas `is.read((char*)&data, sizeof(Data))` : il y a du padding après `a`, et un `std::string` contient un pointeur, pas les caractères.
- **On ne permute pas** `fourcc` ni la chaîne : ce sont des octets isolés, l'endianness ne concerne que les nombres de plus d'un octet.
- Correction des notes : `std::string(taille, ' ')`, le **nombre d'abord**. `string('X', taille)` inverse les arguments : 'X' (code 88) serait pris comme longueur.
- `d.data()` renvoie un `const char*` avant C++17 : on écrit dans `&data.d[0]`.
- Si `taille` vaut 0, on ne lit rien (et `&d[0]` reste valide).

TD2 exercice 2, question 3 : une version portable

Méthode 1 : tester la machine, puis permuter si besoin

```
bool machine_little_endian()
{
  uint16_t v = 0x1234;
  uint8_t* ptr = reinterpret_cast<uint8_t*>(&v);
  return *ptr == 0x34;   // le 1er octet est le
poids faible ?
}
...
if (machine_little_endian()) {  // le fichier est
big endian
  swap_bytes(data.a);
  swap_bytes(data.b);
}
```

C'est le `if (doit_permuter)` des notes de TD : sur une machine big endian, on ne touche à rien.

Méthode 2 : reconstruire avec des décalages

```
uint8_t o[2];
is.read((char*)o, 2);
data.a = int16_t((o[0] << 8) | o[1]); // o[0] =
poids fort

uint8_t p[8];
is.read((char*)p, 8);
uint64_t bits = 0;
for (int i = 0; i < 8; i++)
  bits = (bits << 8) | p[i];              // poids
forts d'abord
std::memcpy(&data.b, &bits, 8);         // <cstring>
```

Les décalages travaillent sur des **valeurs**, pas sur la mémoire : le résultat est le même quel que soit le processeur. Il n'y a même plus de test à faire. Le `memcpy` suppose seulement que le `double` est IEEE 754 des deux côtés (garanti, CM diapo 11).

TD2 exercice 3 : place en mémoire des images

imageur	calcul des pixels	octets	Mio
a. smartphone 4032 × 3024, 24 bits	4032 × 3024 × 3	36 578 304	34,9
b. reflex 7360 × 4912, 24 bits	7360 × 4912 × 3	108 456 960	103,4
c. scanner 512 × 512 × 512, 16 bits	$512^3 \times 2$	268 435 456	256

À cela s'ajoute l'objet lui-même (`sizeof`), négligeable :

objet	PC 32 bits	PC 64 bits
ColorImage : 2 × uint16_t + 1 pointeur	2 + 2 + 4 = **8**	2 + 2 + 4 (padding) + 8 = **16**
volume : 3 × uint16_t + 1 pointeur	6 + 2 (pad.) + 4 = **12**	6 + 2 (pad.) + 8 = **16**

1 Mio = 1024 × 1024 octets (= 1 048 576) ; 1 Mo = 1 000 000 octets. Les notes de TD divisent par 1024 deux fois : ce sont des Mio. Le pointeur fait 4 octets en 32 bits, 8 en 64 bits (CM diapo 10).

Le cas du scanner

Nos classes ne conviennent pas : GrayImage est en **2D** et en **8 bits**. Il faut une classe « volume » avec 3 dimensions et des pixels `uint16_t`, indice **t = (z × height + y) × width + x** (CM diapo 25).

Les dimensions (512) tiennent bien dans un `uint16_t`.

En pratique, le smartphone enregistre un JPEG bien plus petit que ces 34,9 Mio : la photo est compressée avec perte.

TD2 exercice 3 : le débit d'une caméra Ultra HD

Calcul

1 image : 3840 × 2160 pixels × 3 octets = 24 883 200 octets
60 images/s : 24 883 200 × 60 = **1 492 992 000 octets/s**
≈ 1,49 Go/s ≈ **11,9 Gbit/s** (× 8 pour passer en bits)

Attention aux unités : les débits réseau sont en **bits** par seconde, les tailles de fichier en **octets**. 1 octet = 8 bits.

liaison	**débit**	**trop lente de**
5G (théorique)	2,1 Gbit/s	× 5,7
4G/5G (théorique)	615 Mbit/s	× 19
4G/5G (mesuré)	80 à 114 Mbit/s	× 105 à 150
fibre FTTH	100 Mbit/s	× 119
TNT	25 Mbit/s	× 478
ADSL	20 Mbit/s	× 597

Conclusion : aucune liaison grand public ne transporte une vidéo UHD brute. La vidéo est toujours **compressée avec perte** (même principe que JPEG, plus la ressemblance entre images successives).

Images bitmap et classes du cours CM diapos 23 à 28, 38-39

- Image bitmap (raster) = grille de **pixels** ; intensité de 0 (noir) à 255 (blanc), parfois un réel dans [0 ; 1].
- Taille ∝ largeur × hauteur : en $\mathbf{O(n^d)}$ pour une image de côté n en dimension d.
- **Tableau à plat** : pixel (x, y) à l'indice **t = y × width + x** ; en 3D, **t = (z × height + y) × width + x**.
- **Color** = r, g, b en `uint8_t` publics : « TrueColor », 24 bits.
- **ColorImage** = GrayImage avec `Color` au lieu de `uint8_t`. Mieux (mais plus dur) : un modèle `Image<T>` et l'héritage.

élément de GrayImage	pourquoi
`const uint16_t width, height`	la taille ne change jamais : pas de réallocation
`GrayImage() = delete`	une image sans taille n'a pas de sens
`operator=(...) = delete`	l'affectation changerait la taille
constructeur de copie	copie **profonde** (nouveau tableau), sinon double `delete[]`
`new uint8_t[w*h]`	taille connue à l'exécution, trop gros pour la pile
pixels non initialisés	`clear()` ou la lecture le feront
`inline`	éviter le coût d'un appel pour chaque pixel
`uint8_t& pixel(x, y)`	référence : utilisable à gauche de `=`
version `const` de `pixel`	consulter une image constante
`static readPGM`	crée l'image : pas besoin d'une image avant

PGM et PPM : la méthode pour lire un dump hexadécimal TD1, TD2

1. Repérer le **magic number** : `50 35` = "P5" (gris), `50 36` = "P6" (couleur), suivi de `0a`.
2. Une ligne qui commence par 23 ('#') est un **commentaire** jusqu'au `0a` suivant.
3. Les dimensions sont écrites en **caractères** : 33 `32 30 20 32 34 30 0a` = "320 240".
4. `32 35 35 0a` = "255" puis LF : **tout ce qui suit est binaire**.
5. Pixels : 1 octet (P5) ou 3 octets r, g, b (P6), de gauche à droite puis de haut en bas.
6. Vérifier : taille totale = en-tête + w × h (× 3).

Pour P2/P3, tout est en texte (pixels compris, séparés par des blancs) : plus lisible, plus gros, plus lent.

```
// TD1 : les trois fonctions de lecture
void skip_line(std::istream& is)
{
  char c;
  while (is.get(c) && c != '\n') {}
}
void skip_comments(std::istream& is)
{
  char c;
  while (is.get(c) && c == '#')
    skip_line(is);
  if (is) is.putback(c);
}
// readPGM : is >> magic ; skip_blanks ; skip_comments
;
// is >> w >> h ; skip_blanks ; skip_comments ;
// is >> maxval (255) ; new GrayImage(w, h) ;
// is.get() (UN seul blanc) ; is.read(array, w*h)
```

Le détail et les justifications sont dans `TP1_explications.pdf` (pourquoi `static`, pourquoi un seul `get()`, le piège des commentaires après les dimensions).

TD3 : rééchantillonnage, l'essentiel

Plus proche voisin (simpleScale, corrigé du TD3)

On parcourt l'image **d'arrivée** (w × h). Pour chaque (x', y') :

x = x' × width / w **y = y' × height / h** (division entière)

et on recopie `pixel(x, y)`. Résultat : des blocs uniformes.

Exemple

80 × 60 → 823 × 400. Pixel d'arrivée (500, 200) : x = 500 × 80 / 823 = 48, y = 200 × 60 / 400 = 30.

Bilinéaire

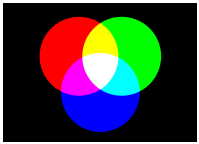
Position réelle (x, y) ; x0 = partie entière, ax = x − x0 (idem en y) :

haut = (1 − ax) P(x0, y0) + ax P(x0+1, y0)
bas = (1 − ax) P(x0, y0+1) + ax P(x0+1, y0+1)
valeur = (1 − ay) haut + ay bas

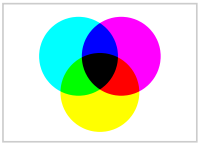
Exemple : P = 10, 20 (haut) et 30, 40 (bas), ax = 0,25, ay = 0,5 → haut 12,5, bas 32,5, valeur **22,5**.

- Aucune information n'est créée : l'image est plus douce, mais floue.
- Correction du demi-pixel : x = (x' + 0,5) × width / w − 0,5.
- Pour une Color : il faut `double * Color` et `Color + Color`.

Les modèles de couleur CM diapos 29 à 37



RGB : additif (écrans)



CMY : soustractif (encres

- **RGB** : on ajoute des lumières ; R + G + B = blanc. Le plus répandu dans les fichiers.
- **CMY(K)** : des encres filtrent la lumière du papier blanc. **C = 1 − R, M = 1 − G, Y = 1 − B** (valeurs dans [0 ; 1]). K (noir) est ajouté car le mélange des encres ne donne pas un vrai noir.
- Les secondaires de RGB (cyan, magenta, jaune) sont les primaires de CMY et inversement.
- Modélisation **par pixel** (tableau de Color, notre choix) ou **par canal** (3 tableaux).

- **YUV** (TV PAL) : Y = **luminance**, U et V = chrominance. **Y = 0,299 R + 0,587 G + 0,114 B**. L'œil distingue mieux les formes en luminance qu'en couleur : intéressant pour compresser (JPEG, MPEG).
- **YIQ** (TV NTSC) : même Y, autre chrominance.
- **HSV** (teinte, saturation, valeur) : proche de la pratique du peintre ; conversion **non linéaire**. HSL : double cône.
- Pros : sRGB, AdobeRGB, CIE XYZ, L*a*b*, plus proches de la perception.
- **Canal alpha** : transparence, de 0 (transparent) à 255 (opaque) ; RGBA ; superposition de calques = alpha-blending.

Exemple : couleur → gris

(200, 100, 50) : Y = 0,299 × 200 + 0,587 × 100 + 0,114 × 50 = 59,8 + 58,7 + 5,7 = **124,2 → 124**.

Palette, pavages, densité CM diapos 40 à 45

Image à palette (codage indirect)

Le pixel ne contient pas la couleur mais un **indice** dans une table de n couleurs RGB. Utilisé quand la mémoire était chère (CGA, VGA 256 couleurs...), et dans GIF, TGA type 1, BMP, PNG.

320 × 240, 256 couleurs : 320 × 240 × 1 + 256 × 3 = 76 800 + 768 = **77 568 o**, contre 230 400 o en 24 bits (≈ 3 fois moins).

16 couleurs : 4 bits par pixel (2 pixels par octet).

Pavages : triangulaire, **carré** (2 sortes de voisins : par un côté, ou par un coin, √2 fois plus loin), hexagonal (tous les voisins à la même distance)... En 3D, le cubique a 3 sortes de voisins.

- **Capteur** : surface perdue entre les photosites, et en général **2 fois plus de sites verts** que de rouges ou de bleus. On fait « comme si » les pixels étaient carrés et complets (99 % du temps).
- **Densité** en ppp (dpi) : pixels par pouce, 1 pouce = 2,54 cm. Écran ≈ 120, imprimante jet d'encre 600 à 1200, laser 2400, scanner 4800.
- **Rapport d'aspect du pixel** = largeur / hauteur d'un pixel (à ne pas confondre avec celui de l'écran, 16:9...). On ne peut pas le deviner à partir de la résolution.

Exemple

Une feuille A4 (21 × 29,7 cm) scannée à 300 ppp : 21 / 2,54 × 300 ≈ **2480** pixels sur 29,7 / 2,54 × 300 ≈ **3508** pixels.

Échantillonnage et quantification

CM diapos 46 à 53

- **Échantillonnage** (spatial) : combien de pixels ? Moins il y en a, moins on voit de détails.
- **Aliasing spatial** : détails plus fins que les pixels → marches d'escalier. Règle : échantillonner au moins au **double de la fréquence** du signal (un pixel au plus tous les demi-détails).
- **Quantification** : combien de niveaux par pixel ? Cela fixe la taille du codage.
- **Bandes de Mach** : un dégradé trop peu quantifié montre des bandes uniformes, renforcées par notre vision (aliasing colorimétrique).
- **Tramage** (Floyd-Steinberg) : on répartit l'erreur sur les voisins, l'image paraît avoir plus de couleurs.
- Trop de niveaux pour l'œil (astronomie : des milliers contre une vingtaine perçus) : **fausses couleurs** ou **fenêtrage**.
- **Z-buffer** (OpenGL, souvent 16 bits) : précision insuffisante → **Z-fighting** (hachures).

apparence	bits / pixel	niveaux
noir et blanc	1	2
4 gris	2	4
16 gris	4	16
256 gris ou 256 couleurs (palette)	8	256
TrueColor	3 × 8 = 24	16,7 millions

Formule à connaître

n bits → 2^n valeurs. Taille brute = largeur × hauteur × bits par pixel / 8 octets.

800 × 600 en 4 bits : 800 × 600 × 4 / 8 = **240 000 octets**.

Les formats d'image CM diapos 54 à 57, 60

Ce qu'on attend d'un format

Échanger entre machines (à l'identique ou avec une perte contrôlée), **indépendamment de l'architecture** (endianness...), lecture rapide, plusieurs quantifications, éventuellement plusieurs images et des métadonnées (auteur, date).

Structure générale

1. **En-tête** : magic number, largeur, hauteur, quantification, compression, présence d'une palette...
2. **Palette** éventuelle : nombre de couleurs, quantification, liste.
3. **Corps** : les pixels, éventuellement compressés.
4. Autres informations éventuelles.

famille	formats
non compressés	gris : PGM, TGA, BMP, TIFF ; couleur : PPM, TGA, BMP, TIFF
sans perte : RLE	TGA, BMP, TIFF
sans perte : Huffman / LZW	GIF, TIFF
sans perte : deflate	PNG
avec perte	JPEG (JFIF), JPEG 2000, TIFF

- **PNM** (PBM, PGM, PPM) : très simple, peu performant, pas de palette ; variante tout texte (P2, P3) ou en-tête texte + pixels binaires (P5, P6).
- **BMP** (Windows, OS/2, aussi pour les icônes) : comparable à TGA ; N&B, gris, couleur, avec ou sans palette, avec ou sans RLE.
- **TIFF** (Adobe) : « méta-format » très général (plusieurs images, couches, JPEG dedans...) : impossible de lire toutes les variantes.
- **Exif / IPTC** : métadonnées (date, ouverture, GPS...) dans JPEG ou TIFF.

Compression sans perte : RLE CM diapos 55 et 59

- **Sans perte** : on retrouve exactement l'image ; on supprime les redondances ; coûte du calcul.
- **Avec perte** : on abîme volontairement (et de façon contrôlée) pour gagner beaucoup.

Taux de compression

taux = taille originale / taille compressée

100 Ko → 20 Ko : taux de **5 : 1**.

RLE : Run Length Encoding

- Suite de valeurs **identiques** → (nombre, valeur) : on gagne.
- Suite de valeurs **différentes** → (nombre négatif, valeurs) : on perd un peu.
- Efficace quand il y a **peu de couleurs** ; simple mais peu efficace en général (photos).

L'exemple du CM

`A B C D D D D E E E E F G (14 symboles)`

`−3 A B C | 5 D | 4 E | −2 F G (11 symboles)`

−3 : « 3 valeurs différentes suivent » ; 5 D : « D répété 5 fois ».

La version TGA (TP4)

Un octet d'en-tête par paquet : bit 7 = 1 → paquet répété, bit 7 = 0 → paquet brut ; les 7 autres bits = nombre − 1 (au plus 128 pixels).

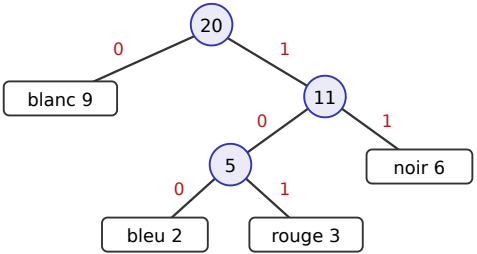
R R R R V B B → 83 R, 00 V, 81 B : 3 + 3 × 3 = 12 octets au lieu de 21.

Chiffres réels du TP4 : une photo passe de 230 418 à 207 106 octets (1,1 : 1) ; une image de texte sur fond uni de 242 208 à 11 985 octets (20 : 1).

Compression sans perte : Huffman, LZW

CM diapos 61 à 65

Huffman : les couleurs fréquentes reçoivent des codes **courts**, les rares des codes longs. Exemple : 20 pixels, blanc × 9, noir × 6, rouge × 3, bleu × 2.



1. Histogramme (fréquences ci-contre).
2. On regroupe les **deux moins fréquents** : bleu 2 + rouge 3 = 5.
3. On recommence : 5 + noir 6 = 11, puis blanc 9 + 11 = 20 (la racine = nombre de pixels).
4. Branche gauche = 0, droite = 1 ; code = chemin depuis la racine.

couleur	code	bits
blanc	0	9 × 1
noir	11	6 × 2
bleu	100	2 × 3
rouge	101	3 × 3

36 bits, contre 40 avec un code fixe de 2 bits. Aucun code n'est le début d'un autre : le décodage est sans ambiguïté.

LZW (Lempel-Ziv-Welch) : cherche des **motifs** qui se répètent (efficace même sur des images tramées) ; utilisé par **GIF** (brevet UNISYS jusqu'en 2004).

GIF, PNG, JPEG, JPEG 2000 CM diapos 63 à 78

- **GIF (89a)** : LZW ; gris 8 bits ou **palette de 256 couleurs** parmi 16,7 M ; animations ; transparence tout ou rien (une couleur sacrifiée). Pour logos, schémas.
- **PNG** (1995, ouvert) : remplace GIF ; compression sans perte **deflate** (zlib) ; gris, couleurs, palette, **canal alpha**, affichage progressif. Animations : MNG, APNG.
- **Entrelacement Adam7** : les pixels sont envoyés en passes de résolution croissante ; on affiche une image grossière qui s'affine pendant le téléchargement.
- **JPEG 2000** : bien meilleur à forte compression, pas de blocs, moins d'artefacts... mais peu utilisé. Les pros préfèrent le **RAW** sans perte.

JPEG (ISO 10918-1, 1994)

Pour les **photos** ; mal adapté aux schémas nets et aux images à peu de couleurs. Perte réglée par la qualité. Bibliothèque libjpeg (TP3).

1. RGB → **YCbCr** ;
2. **sous-échantillonnage** de Cb, Cr (perte) ;
3. blocs **8 × 8**, puis **DCT** ;
4. **quantification** des coefficients (perte, réglée par la qualité) ;
5. parcours en **zigzag**, **RLE des 0** avec marque de fin ;
6. **Huffman**.

Chiffres du CM : 769 Ko → 200 Ko à 99 % (4 : 1), 52 Ko à 75 % (15 : 1), 6 Ko à 1 % (128 : 1).

Images vectorielles CM diapos 81 à 86

- On stocke **la façon de dessiner** (segments, courbes, formes), pas des pixels.
- Qualité **indépendante de la résolution** (on peut zoomer), souvent très compact (PDF 37 Ko contre PNG 156 Ko dans le CM).
- Il faut retracer toutes les primitives à chaque affichage.
- Parfait pour le symbolique (texte, schémas) ; **inadapté aux photos**.

	bitmap	**vectoriel**
contenu	grille de pixels	primitives
zoom	pixels visibles	net
photo	oui	non
exemples	PNG, JPEG, TGA	SVG, PDF, PostScript

- **Polices** : PostScript type 1 (courbes de Bézier **cubiques**) ; TrueType (courbes **quadratiques**). TTF → PS1 sans perte, pas l'inverse. Polices bitmap (.FON) pour l'écran.
- **SVG** : XML (arbre), norme du W3C, alpha, dégradés, animation en JavaScript ; icônes Gnome/KDE ; Inkscape.
- **PostScript** : langage de description de page en notation polonaise inversée, interprété par les imprimantes laser.
- **PDF** : successeur pour les documents ; norme ISO (2008), polices incorporées, compression, formulaires, liens, chiffrement.
- **TeX/LaTeX** : composition typographique (le CM est fait avec Beamer).
- **3D** : surfaces triangulées, NURBS... (.obj, .dxf, VRML, X3D).

Exercices d'entraînement (1/2)

1. Combien d'octets occupe en mémoire `struct S { char a; int b; char c; };` ? Et `struct T { int b; char a; char c; };` ?
2. Écrire les octets de l'entier 16 bits 0x0A0B en little endian puis en big endian.
3. Sur un PC, après `uint32_t v = 0x11223344; uint8_t* p = (uint8_t*)&v;`, que vaut `p[0]` ? `p[3]` ?
4. Que vaut `v` après `swap_bytes(v)` ?
5. Combien de tours fait `for (uint8_t i = 250; i < 255; i++)` ? Et avec `i <= 255` ?
6. Quelle est la taille d'un fichier PPM P6 de 640 × 480 dont l'en-tête fait 15 octets ?
7. Décoder ce début de fichier :
   `50 35 0a 32 20 33 0a 32 35 35 0a 00 ff 80 80 ff 00`
8. Dans `chat.pgm` (320 × 240), à quel indice de `array` se trouve le pixel (10, 2) ?
9. Image 100 × 100 en 16 couleurs avec palette 24 bits : taille des pixels et de la palette ?
10. Taux de compression d'une image de 1,2 Mo devenue 80 Ko ?
11. Coder en RLE (notation du CM) : `X X X X Y Z Z Z`.
12. Construire les codes de Huffman pour : A × 8, B × 4, C × 2, D × 2. Combien de bits au total ?
13. Agrandir une image 4 × 4 en 10 × 10 par plus proche voisin : quel pixel source pour (x', y') = (7, 3) ?

Exercices d'entraînement (2/2) : corrigés

1. S : 1 + 3 (pad.) + 4 + 1 + 3 (pad. final) = **12**. T : 4 + 1 + 1 + 2 = **8**.
2. Little : `0B 0A` ; big : `0A 0B`.
3. `p[0]` = 0x44 (poids faible en premier), `p[3]` = 0x11.
4. 0x44332211.
5. 5 tours (250 à 254). Avec `<= 255` : boucle **infinie** (255 + 1 = 0).
6. 15 + 640 × 480 × 3 = 15 + 921 600 = **921 615** octets.
7. "P5", largeur 2, hauteur 3, maxval 255, puis 6 pixels : (0, 255) / (128, 128) / (255, 0).
8. 2 × 320 + 10 = **650**.
9. 16 couleurs = 4 bits : 100 × 100 × 4 / 8 = **5 000** octets ; palette 16 × 3 = **48** octets.
10. 1 200 / 80 = **15 : 1** (en Ko des deux côtés).
11. **4 X | −1 Y | 3 Z** : 8 symboles → 6 (le Y isolé coûte 2 symboles).
12. C + D = 4 ; 4 + B = 8 ; 8 + A = 16. Codes : A = 0, B = 11, C = 100, D = 101 (selon le côté choisi). Bits : 8 × 1 + 4 × 2 + 2 × 3 + 2 × 3 = **28** (contre 32 avec 2 bits fixes).
13. x = 7 × 4 / 10 = 2 (2,8 tronqué), y = 3 × 4 / 10 = 1 : pixel **(2, 1)**.

Ce que le prof n'a pas corrigé au tableau

- **TD3, partie bilinéaire** : le corrigé s'arrête au prototype (`TP3_TD3/corrige_prof.txt`), le prof a demandé de chercher à partir de ce qui a été vu. La formule de cette fiche et des explications du TP2 est la formule bilinéaire standard : deux interpolations horizontales puis une verticale. C'est aussi celle qu'attend le sujet du TP2 (« ne pas changer la formule vue en TD sur des niveaux de gris »).
- La **correction du demi-pixel** est un plus proposé par le sujet du TP2 ; sans elle, on a « pratiquement tous les points ».
- Les **figures du CM** (arbre de Huffman diapo 61, schémas JPEG) sont à relire directement dans le PDF du cours.

Où trouver quoi

- Explications de chaque TP : `TPx/TPx_explications.pdf`.
- Code testé : `PROJET_FINAL/` (`make test`).
- Sujets et corrigés pris en TD : dans chaque dossier `TPx`.